

Neural Networks from Scratch and Exploring Compression

Robert Davidson

October 5, 2026

1 Feed Forward Neural Networks

1.1 Motivation

Given some unknown function, $f^* : X \rightarrow Y$, we aim to find a good approximation, $f : X \rightarrow Y$, by learning a set of parameters, θ , using a dataset of N samples:

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i = f^*(\mathbf{x}_i))\}_{i=1}^N$$

We measure how well our approximation performs by measuring the loss, $L(\mathbf{y}_i, \hat{\mathbf{y}}_i)$. We then optimize the parameters, θ , by minimizing the loss over the dataset.[1][3]

1.2 Neurons and Layers

A neural network is a function, $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ takes an input vector $\mathbf{x} \in \mathbb{R}^n$ and produces an output vector $\hat{\mathbf{y}} \in \mathbb{R}^m$ by passing the input through a series of layers of neurons.

The neuron is the basic building block of a neural network. When we pass a vector to the network, the neuron computes a weighted sum of the inputs and adds a bias term,

$$a = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T \mathbf{x} + b$$

This is commonly called the preactivation value. We then apply a non linear activation function, σ to obtain the output of the neuron:

$$h = \sigma(a)$$

We commonly taken $\sigma = \text{ReLU}$, where $\text{ReLU}(x) = \max(0, x)$.

We say a layer is a series of neurons that take the same input, apply their own unique weights and biases, and product a vector of outputs. For example, the first hidden layer of a neural network can be written:

$$\mathbf{h}^{(1)} = \begin{bmatrix} h_1^{(1)} \\ h_2^{(1)} \\ \vdots \\ h_m^{(1)} \end{bmatrix} = \begin{bmatrix} \sigma(w_{1,1}^{(1)}x_1 + w_{1,2}^{(1)}x_2 + \dots + w_{1,n}^{(1)}x_n + b_1^{(1)}) \\ \sigma(w_{2,1}^{(1)}x_1 + w_{2,2}^{(1)}x_2 + \dots + w_{2,n}^{(1)}x_n + b_2^{(1)}) \\ \vdots \\ \sigma(w_{m,1}^{(1)}x_1 + w_{m,2}^{(1)}x_2 + \dots + w_{m,n}^{(1)}x_n + b_m^{(1)}) \end{bmatrix} = \sigma \left(\begin{bmatrix} (w^{(1)})^T \\ (w^{(1)})^T \\ \vdots \\ (w^{(1)})^T \end{bmatrix} \right) = \sigma(W^{(1)}\mathbf{x} + \mathbf{b})$$

The second hidden layer takes this vector as input and produces another vector of outputs:

$$\mathbf{h}^{(2)} = \sigma(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

In general, the l -th layer of a neural network can be written as:

$$\mathbf{h}^{(l)} = \sigma(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

1.3 Questions

We might ask ourselves, why do we need to use a non linear activation function? How do we know our choice of weights is correct?

2 MNIST and Our Neural Network

Mnist is a dataset of 28×28 pixel images of handwritten digits. Each image is labeled with the digits it represents from 0 to 9. Let $\mathcal{D}$ denote the dataset, then:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{60,000}$$

where $x_i \in \mathbb{R}^{28 \times 28}$ and $y_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

We use one hidden layer, with 256 neurons, and a ReLU activation function. Our network can be written as:

$$\hat{y} = f(x) = \text{softmax}(W^{(2)}(\text{ReLU}(W^{(1)}x + b^{(1)}) + b^{(2)}))$$

with:

$$W_1 \in \mathbb{R}^{256 \times 784}, b_1 \in \mathbb{R}^{256}, W_2 \in \mathbb{R}^{10 \times 256}, b_2 \in \mathbb{R}^{10}$$

We initialize the weights and biases following Kaiming He's initialization scheme.

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [2] Warren S. McCulloch and Walter Pitts. (1943) warren s. mcculloch and walter pitts, "a logical calculus of the ideas immanent in nervous activity," *bulletin of mathematical biophysics* 5: 115-133. In James A. Anderson and Edward Rosenfeld, editors, *Neurocomputing, Volume 1: Foundations of Research*. The MIT Press, 04 1988.
- [3] Bart M. N. Smets. *Mathematics of neural networks*, 2024.
- [4] Wikipedia contributors. Artificial neuron — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Artificial_neuron&oldid=1378551412, 2026. [Online; accessed 5-October-2026].